

2022年 5月 03日

# 동적계획법 기반 저수지 운영

김연주 / 이재황 / 이재호 / 박성열

수자원 System 공학

# [발표순서]

1

수리 Model 개요

2

전산모의 Code 및 결과 논의

3

전산모의 결과

4

최적화 기법 별 특성 / 한계점 비교

# [수리 Model 개요] 목적함수 설정

$$\text{Minimize } \sum_t TSD_t$$

$$TSD = ws_t[(TS - S_t)^2 + (TS - S_{t+1})^2] + wf_t[(ES)^2 + (ES_{t+1})^2] + wr_t[DR_t^2]$$

목표 저수량 대비 실제  
저수량에 대한 편차 정의

초과 저수량으로부터  
발생한 편차 정의

부족 저수량으로부터  
발생한 편차 정의

# [수리 Model 개요] 공통 제약조건 설정 – 방류량 제약

## ■ 각 시점별 방류량 제약조건을 다르게 설정

- $S_t \leq TF + ES_t, \forall t \in \{1,2\}$

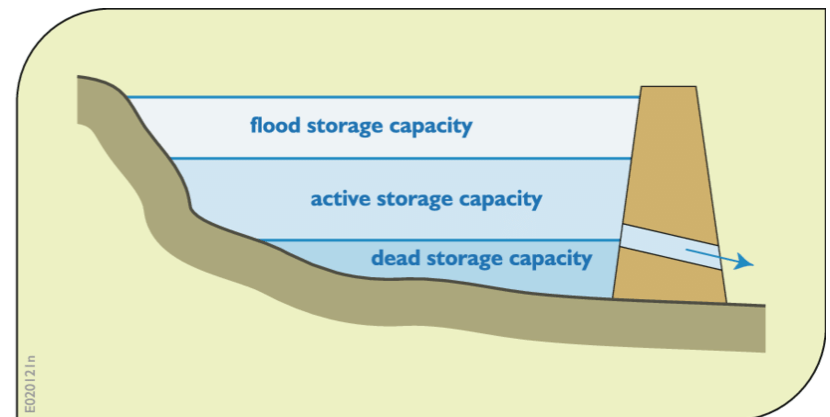
⇒ 초기 1차/2차년도 별 저수량 상태에 대한 제약조건

⇒ 매  $t$  시점에서의 저수량 계산 활용 변수는 홍수조절 목적의 저수제한량과 초과 저수량으로 제한

- $R_t \geq TR_t - DR_t, \forall t \in T$

⇒ 실제 방류량은 목표 방류량에서 부족 방류량을 제외한 값과 같거나 더 많이 방류할 것으로 제약

⇒ 매 Scheduling Period에 적용되는 제약조건



# [수리 Model 개요] 공통 제약조건 설정 – 저수량 제약조건

## ■ 저수지 입력 및 출력에 대한 제약조건 설정

- $S_{t+1} = S_t + Q_t - R_t - L_t(S_t, S_{t+1})$  및  $S_t \leq K$

⇒ 매  $t$  시점에서의 저수량은  $t$  시점에서의 유입량, 방류량 및 저수지 내 손실량에 따라 결정

⇒ 매  $t$  시점별 저수량은 저수지 용량  $K$ 와 같거나 작을 것을 제약

# [수리 Model 개요] 제약조건 설정 – Search 논리 설정 (1/4)

## ■ Backward-moving Solution 적용 – 첫 Scheduling Period

$$F_t^n(S_t, Q_t) = \min \sum_{t=1,n} TSD_t(S_t, R_t, S_{t+1})$$

$$TSD = w_{s_t}[(TS - S_t)^2 + (TS - S_{t+1})^2] + w_{f_t}[(ES)^2 + (ES_{t+1})^2] + w_{r_t}[DR_t^2]$$

## ■ 하기 제약조건들을 준수하면서, Period-wise Decision을 결정

$$S_t \leq TF + ES_t, \quad \forall t \in \{1,2\}$$

$$S_t \leq K, \quad \forall t \in T$$

$$S_{t+1} = S_t + Q_t - R_t - L_t(S_t, S_{t+1})$$

$$R_t \geq TR_t - DR_t, \quad \forall t \in T$$

# [수리 Model 개요] 제약조건 설정 – Search 논리 설정 (2/4)

## ■ 차기 Scheduling Period에서의 Decision Making 절차

$$F_t^n(S_t, Q_t) = \text{minimum} \{ \underbrace{TSD_t(S_t, R_t, S_{t+1})}_{\text{현재 단계에서 고려될 최소 오차 발생 가능 Schedule에 대한 조합 (방류량 및 저수량)}} + \underbrace{F_{t+1}^{n-1}(S_{t+1}, Q_{t+1})}_{\text{전 단계에서 고려된 최소 오차 발생 가능 Schedule에 대한 조합 (저수량 및 유입량)}} \}$$

현재 단계에서 고려될  
최소 오차 발생 가능  
Schedule에 대한 조합  
(방류량 및 저수량)

전 단계에서 고려된  
최소 오차 발생 가능  
Schedule에 대한 조합  
(저수량 및 유입량)

# [수리 Model 개요] 제약조건 설정 – Search 논리 설정 (3/4)



## 차기 Scheduling Period에서의 제약조건 – 방류량이 결정변수

- $0 \leq S_t + Q_t - L_t(S_t, S_{t+1}) - R_t$

⇒ [하한값 설정]

- 각 시점별 최소 방류 가능량에 대한 조합은 저수량의 하한선 (0) 보다 큰 선에서만 가능

- $20 \geq S_t + Q_t - R_t - L_t$

⇒ [상한값 설정]

- 각 시점별 최대 방류 가능량에 대한 조합은 최대 저수량을 넘지 않는 선에서만 가능

- $S_{t+1} = S_t + Q_t - R_t - L_t(S_t, S_{t+1})$

⇒ [Balance 설정]

- 방류 및 유입량에 따른 저수량의 변화 설정

# [수리 Model 개요] 제약조건 설정 – Search 논리 설정 (4/4)



## ■ 차기 Scheduling Period에서의 제약조건 – 저수량이 결정변수

- $0 \leq S_{t+1} \leq 20$

⇒ [상/하한값 설정]

- 차기 시점별 저수량은 최대 및 최소 저수량 허용량을 준수하는 범위에서만 조합 가능

- $R_t = S_t + Q_t - S_{t+1} - L_t(S_t, S_{t+1})$

⇒ [방류량 제약조건 설정]

- 각 시점별 최대 방류 가능량에 대한 조합은 최대 저수량을 넘지 않는 선에서만 가능

- $S_{t+1} \leq S_t + Q_t - R_t - L_t(S_t, S_{t+1})$

⇒ [Balance 설정]

- 방류 및 유입량에 따른 저수량의 변화 설정

# 전산모의 Code 및 결과 논의

# [전산모의] 정식화 대비 전산모의 구현 비교 (1/10)

## ■ Input Parameter 선언

- [방류량/손실량/유입량] 각 변수 별  $t$  시점별 주어진 값을 Scenario를 3차원 행렬로 구현  
⇒ Textbook에 나온 기본 Matrix와 동일하게 구현
  - 방류량(5,5,4) / 손실량(5,5,4) / 유입량(5,5,4) / 목표저수량 (5,5,4) / 가중치 (5,5,4)

### <방류량 입력값>

| , , t=1 |    |    |    |    |    |
|---------|----|----|----|----|----|
|         | 0  | 5  | 10 | 15 | 20 |
| 0       | 24 | 19 | 14 | 9  | 3  |
| 5       | 29 | 24 | 19 | 13 | 8  |
| 10      | 34 | 29 | 23 | 18 | 13 |
| 15      | 39 | 33 | 28 | 23 | 18 |
| 20      | 44 | 38 | 33 | 28 | 23 |

 ... 

| , , t=4 |    |    |    |    |    |
|---------|----|----|----|----|----|
|         | 0  | 5  | 10 | 15 | 20 |
| 0       | 18 | 13 | 8  | 3  | 0  |
| 5       | 23 | 18 | 13 | 8  | 2  |
| 10      | 28 | 23 | 18 | 12 | 7  |
| 15      | 33 | 28 | 22 | 17 | 12 |
| 20      | 38 | 32 | 27 | 22 | 17 |

### <손실량 입력값>

| , , t=1 |     |     |     |     |     |
|---------|-----|-----|-----|-----|-----|
|         | 0   | 5   | 10  | 15  | 20  |
| 0       | 0.0 | 0.1 | 0.3 | 0.4 | 0.6 |
| 5       | 0.1 | 0.2 | 0.4 | 0.6 | 0.8 |
| 10      | 0.3 | 0.4 | 0.6 | 0.8 | 1.0 |
| 15      | 0.4 | 0.6 | 0.8 | 1.0 | 1.2 |
| 20      | 0.5 | 0.8 | 1.0 | 1.2 | 1.4 |

 ... 

| , , t=4 |     |     |     |     |     |
|---------|-----|-----|-----|-----|-----|
|         | 0   | 5   | 10  | 15  | 20  |
| 0       | 0.0 | 0.1 | 0.2 | 0.3 | 0.4 |
| 5       | 0.1 | 0.2 | 0.3 | 0.4 | 0.5 |
| 10      | 0.2 | 0.3 | 0.4 | 0.5 | 0.6 |
| 15      | 0.3 | 0.4 | 0.5 | 0.6 | 0.7 |
| 20      | 0.4 | 0.5 | 0.6 | 0.7 | 0.8 |

### <유입량 입력값>

| , , t=1 |    |    |    |    |    |
|---------|----|----|----|----|----|
|         | 0  | 5  | 10 | 15 | 20 |
| 0       | 24 | 24 | 24 | 24 | 24 |
| 5       | 24 | 24 | 24 | 24 | 24 |
| 10      | 24 | 24 | 24 | 24 | 24 |
| 15      | 24 | 24 | 24 | 24 | 24 |
| 20      | 24 | 24 | 24 | 24 | 24 |

 ... 

| , , t=4 |    |    |    |    |    |
|---------|----|----|----|----|----|
|         | 0  | 5  | 10 | 15 | 20 |
| 0       | 18 | 18 | 18 | 18 | 18 |
| 5       | 18 | 18 | 18 | 18 | 18 |
| 10      | 18 | 18 | 18 | 18 | 18 |
| 15      | 18 | 18 | 18 | 18 | 18 |
| 20      | 18 | 18 | 18 | 18 | 18 |

# [전산모의] 정식화 대비 전산모의 구현 비교 (2/10)

## ■ Variable 선언 (목적함수 TSD 내 변수) – 오차저수량은 Variable로 구분

### • $[DR_t]$

⇒ 각 Period 별 Variable의 범위 설정 목적의 Initial Base Matrix 생성

- 매 시점별 목표저수량과 방류량 시나리오 (Input Parameter)의 차로 설정
- $DR_t$  역시, Positive Variable로 구분하여 발생한 Negative 값들에 대하여 0 처리

```
DR_t <- array(  
  target_TR-R,  
  dim=c(5,5,4),  
  dimnames = list(c(0,5,10,15,20),c(0,5,10,15,20),c("t=1","t=2","t=3","t=4"))  
)  
  
DR_t[is.negative(DR_t)] <- 0
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (3/10)

## ■ 제약조건 선언 – t 시점별 방류 가능량에 대한 정의

- $[R_t^{const} = TR_t - DR_t]$

⇒ 각 Period 별  $R_t^{const}$ 에 대한 범위값 저장을 위한 Initial Base Matrix 생성

- 각 시점별 실제 방류량에 대한 범위 설정 (목표방류량과 부족방류량의 차로 설정)

```
# 제약조건
#Rt >= TR_t - DR_t
Rt_constraint <- array(
  target_TR-DR_t,
  dim=c(5,5,4),
  dimnames = list(c(0,5,10,15,20),c(0,5,10,15,20),c("t=1","t=2","t=3","t=4"))
)
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (4/10)

## ■ 제약조건 선언 – t 시점별 방류 가능량에 대한 정의

- $[R_t^{sc} \geq R_t^{const}]$

⇒ 실제 방류량에 대한 수치적 범위 설정

- 입력값의 실제 방류량이 제약조건을 부합하는 값일 경우, 해당 값을 가능 Scenario로 설정
- 해당 조건을 준수하지 않을 경우, 제약조건에서 도출된 값을 가능 Scenario로 설정
- 다만, 방류량 역시 Positive Variable로 취급함에, Negative 값들의 경우 0 처리

```
for (i in 1:5){  
  for (j in 1:5){  
    for (k in 1:4){  
      if(R[i,j,k] >= Rt_constraint[i,j,k]){  
        R_applied[i,j,k]<-R[i,j,k]  
      }else{  
        R_applied[i,j,k] <- Rt_constraint[i,j,k]  
      }  
    }  
  }  
}
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (5/10)

## ■ 제약조건 선언 – t 시점별 발생 가능 초과저수량에 대한 정의

- $[S_t \leq TF + ES_t, \forall t \in \{1,2\}]$

⇒ 각 시점별 저수량에서 수위조절저수량의 차를 발생 가능한 초과저수량의 범위로 설정

- 다만, 시점별 목표저수량과 수위조절저수량의 값이 동일함에, 목표저수량으로 취급

$$(ES_t = \text{초기저수량} - \text{수위조절저수량})$$

```
ES_t <- array(  
  initial_storage-target_TS,  
  dim=c(5,5,4),  
  dimnames = list(c(0,5,10,15,20),c(0,5,10,15,20),c("t=1","t=2","t=3","t=4"))  
)  
  
ES_t[is.negative(ES_t)] <- 0
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (6/10)

## Variable 선언 (목적함수 TSD 내 변수) – 초과저수량은 Variable로 구분

- $[ES_{t+1}]$

⇒ 각 Period 별  $ES_{t+1}$  범위값 저장을 위한 Initial Base Matrix 생성

- 최종저수량과 각 시점별 수위조절저수량의 차로 설정 ( $ES_{t+1} = \text{최종저수량} - \text{수위조절저수량}$ )
- $ES_{t+1}$ 의 경우, 미래값에 대한 Scenario를 고려함에, Final Storage값과의 차를 적용
- $ES_{t+1}$  또한 Positive Variable로, Negative 값을 0 처리

```
ES_tplus1 <- array(  
  final_storage-target_TS,  
  dim=c(5,5,4),  
  dimnames = list(c(0,5,10,15,20),c(0,5,10,15,20),c("t=1","t=2","t=3","t=4"))  
)  
  
ES_tplus1[is.negative(ES_tplus1)] <- 0
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (7/10)

## Variable 선언 (TSD)

- [TSD] 발생 가능 TSD에 대한 Scenario 생성

⇒ 각 변수 및 가중치 조합에 따라 발생 가능한 오차를 계산

- $S_t$  및  $S_{t+1}$  에 대한 기본 Scenario는 초기저수량과 최종저수량을 적용

```
#calculate TSD
TSD_with_negative <- array(
  NA,
  dim=c(5,5,4),
  dimnames = list(c(0,5,10,15,20),c(0,5,10,15,20),c("t=1", "t=2", "t=3", "t=4"))
)

for(i in 1:5){
  for(j in 1:5){
    for(t in 1:4){
      TSD_with_negative[i,j,t] <- ws_t[i,j,t]*((target_TS[i,j,t]-initial_storage[i,j,t])^2+(target_TS[i,j,t]-final_storage[i,j,t])^2)+wf_t[i,j,t]*(ES_t[i,j,t]^2+ES_tplu
    }
  }
}

TSD <- TSD_with_negative
TSD[is.negative(R_with_negative)] <- NaN
print(TSD)
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (8/10)

## ■ Variable 선언 (TSD)

- [TSD\_Optimized] 발생 가능 TSD에 대한 Scenario 생성
  - ⇒ n : 시작점
  - ⇒ k : 총 Iteration 횟수 (Textbook의 경우 10번 지시)

```
n <- 2
k <- 30

#n = ?????????????? ???????, year??? ???????
TSD_optimized <- array(NA,dim = c(5,5,k))
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (9/10)

## ■ Path Search 논리 구현

- Backward Recursion 적용 (시간 Matrix를 4,3,2,1순으로 생성)

$$F_t^n(S_t, Q_t) = \text{minimum} \{TSD_t(S_t, R_t, S_{t+1}) + F_{t+1}^{n-1}(S_{t+1}, Q_{t+1})\}$$

```
n <- 2
k <- 30

#n = ?????????????? ??????, year??? ??????
TSD_optimized <- array(NA,dim = c(5,5,k))

while(n <= k){
  t = rep(seq(4,1,by=-1))[(n-1)%4+1]
  tminus1 = rep(seq(4,1,by=-1))[(n+2)%4+1]
  for(i in 1:5){
    for(j in 1:5){
      TSD_optimized[, ,1] <- TSD[, ,4]
      TSD_optimized[i,j,n] <- TSD[i,j,t] + min(TSD_optimized[j, ,n-1], na.rm = T)
    }
  }
  if(min(TSD_optimized[1, ,n-1], na.rm = T) - min(TSD_optimized[1, ,n], na.rm = T) == 186){
    break
  }
  n <- n+1
}
```

# [전산모의] 정식화 대비 전산모의 구현 비교 (10/10)

## ■ 전산모의 결과값 저장

- 각 (Scheduling Interval , 변수 개수 , Iteration 횟수)

⇒ 변수 개수 : 조합 가능 최종저수량 + Iteration 별 저장공간 + 현시점 방류량 + 전시점 저수량

```
simulation <- array(  
  NA,  
  dim=c(4,8,k),  
  dimnames = list(c(1,2,3,4),c(1,2,3,4,5,6,7,8),c(seq(1,k,1)))  
)  
  
for(j in 1:5){  
  for(i in 1:k){  
    simulation[,1,i] <- i  
    simulation[,2,i] <- rep(c(1,2,3,4))  
    #simulation[j,3,i] <- initial_storage[,simulation[j,2,i]]  
    simulation[,4,i]  
    simulation[,5,i]  
  }  
}  
  
print(simulation)
```

# 전산모의 결과

# [결과분석] 목적함수 값 비교 – 기본값 적용 (Case1)

## Scheduling Period 별 TSD값 변화추이

- Textbook에 기재된 Solution의 값과 동일하게 도출

⇒ [전제조건]  $Q(t) = 24, 12, 6, 18$  (for  $t = 1, 2, 3, 4$ )

, , 2

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 996  | 1025 | NaN  | NaN  | NaN  |
| [2,] | 725  | 675  | 725  | NaN  | NaN  |
| [3,] | 525  | 425  | 425  | 525  | NaN  |
| [4,] | 425  | 275  | 225  | 306  | NaN  |
| [5,] | 400  | 225  | 136  | 146  | 256  |

, , 3

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1805 | 1381 | 1121 | NaN  | NaN  |
| [2,] | 1621 | 1141 | 831  | 671  | NaN  |
| [3,] | 1496 | 1000 | 641  | 431  | 432  |
| [4,] | 1421 | 925  | 550  | 291  | 261  |
| [5,] | 1396 | 900  | 525  | 250  | 161  |

...

, , 6

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1190 | 1215 | NaN  | NaN  | NaN  |
| [2,] | 919  | 865  | 911  | NaN  | NaN  |
| [3,] | 719  | 615  | 611  | 711  | NaN  |
| [4,] | 619  | 465  | 411  | 492  | NaN  |
| [5,] | 594  | 415  | 322  | 332  | 442  |

...

, , 10

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1376 | 1401 | NaN  | NaN  | NaN  |
| [2,] | 1105 | 1051 | 1097 | NaN  | NaN  |
| [3,] | 905  | 801  | 797  | 897  | NaN  |
| [4,] | 805  | 651  | 597  | 678  | NaN  |
| [5,] | 780  | 601  | 508  | 518  | 628  |

period  $t = 3$   $n = 2$   $Q_3 = 6$   
 $F_3^2(S_2, Q_3) = \min \{TSD_3(S_2, R_3, S_4) + F_2^1(S_2, Q_2)\}$

| initial storage $S_3$ | TSD <sub>3</sub> + $F_2^1(S_2, Q_2)$<br>final storage $S_4$ |      |     |     |     | $F_3^2(S_2, Q_3)$ | $R_3$ | $S_4$ |
|-----------------------|-------------------------------------------------------------|------|-----|-----|-----|-------------------|-------|-------|
|                       | 0                                                           | 5    | 10  | 15  | 20  |                   |       |       |
| 0                     | 996                                                         | 1025 | -   | -   | -   | 996               | 6     | 0     |
| 5                     | 725                                                         | 675  | 725 | -   | -   | 675               | 5     | 5     |
| 10                    | 525                                                         | 425  | 425 | 525 | -   | 425               | 5-10  | 5-10  |
| 15                    | 425                                                         | 275  | 225 | 306 | -   | 225               | 10    | 10    |
| 20                    | 400                                                         | 225  | 136 | 146 | 256 | 136               | 14    | 10    |

period  $t = 2$   $n = 3$   $Q_2 = 12$   
 $F_2^1(S_2, Q_2) = \min \{TSD_2(S_2, R_2, S_3) + F_1^0(S_2, Q_1)\}$

| initial storage $S_2$ | TSD <sub>2</sub> + $F_1^0(S_2, Q_1)$<br>final storage $S_3$ |         |         |         |         | $F_2^1(S_2, Q_2)$ | $R_2$ | $S_3$ |
|-----------------------|-------------------------------------------------------------|---------|---------|---------|---------|-------------------|-------|-------|
|                       | 0                                                           | 5       | 10      | 15      | 20      |                   |       |       |
| 0                     | 809+996                                                     | 689+675 | 696+425 | -       | -       | 1121              | 1     | 10    |
| 5                     | 625+996                                                     | 466+675 | 406+425 | 446+225 | -       | 671               | 1     | 15    |
| 10                    | 500+996                                                     | 325+675 | 216+425 | 206+225 | 296+136 | 431               | 6     | 15    |
| 15                    | 425+996                                                     | 250+675 | 125+425 | 66+225  | 125+136 | 261               | 5     | 20    |
| 20                    | 400+996                                                     | 225+675 | 100+425 | 25+225  | 25+136  | 161               | 10    | 20    |

period  $t = 3$   $n = 6$   $Q_3 = 6$   
 $F_3^2(S_2, Q_3) = \min \{TSD_3(S_2, R_3, S_4) + F_2^1(S_2, Q_2)\}$

| initial storage $S_3$ | TSD <sub>3</sub> + $F_2^1(S_2, Q_2)$<br>final storage $S_4$ |          |         |         |         | $F_3^2(S_2, Q_3)$ | $R_3$ | $S_4$ |
|-----------------------|-------------------------------------------------------------|----------|---------|---------|---------|-------------------|-------|-------|
|                       | 0                                                           | 5        | 10      | 15      | 20      |                   |       |       |
| 0                     | 996+194                                                     | 1025+190 | -       | -       | -       | 1190              | 6     | 0     |
| 5                     | 725+194                                                     | 675+190  | 725+186 | -       | -       | 865               | 5     | 5     |
| 10                    | 525+194                                                     | 425+190  | 425+186 | 525+186 | -       | 611               | 5     | 10    |
| 15                    | 425+194                                                     | 275+190  | 225+186 | 306+186 | -       | 411               | 10    | 10    |
| 20                    | 400+194                                                     | 225+190  | 136+186 | 146+186 | 256+186 | 322               | 14    | 10    |

period  $t = 3$   $n = 10$   $Q_3 = 6$   
 $F_3^{10}(S_2, Q_3) = \min \{TSD_3(S_2, R_3, S_4) + F_2^1(S_2, Q_2)\}$

| initial storage $S_3$ | TSD <sub>3</sub> + $F_2^1(S_2, Q_2)$<br>final storage $S_4$ |          |         |         |         | $F_3^{10}(S_2, Q_3)$ | $R_3$ | $S_4$ |
|-----------------------|-------------------------------------------------------------|----------|---------|---------|---------|----------------------|-------|-------|
|                       | 0                                                           | 5        | 10      | 15      | 20      |                      |       |       |
| 0                     | 996+380                                                     | 1025+376 | -       | -       | -       | 1376                 | 6     | 0     |
| 5                     | 725+380                                                     | 675+376  | 725+372 | -       | -       | 1051                 | 5     | 5     |
| 10                    | 525+380                                                     | 425+376  | 425+372 | 525+372 | -       | 797                  | 5     | 10    |
| 15                    | 425+380                                                     | 275+376  | 225+372 | 306+372 | -       | 597                  | 10    | 10    |
| 20                    | 400+380                                                     | 225+376  | 136+372 | 146+372 | 256+372 | 508                  | 14    | 10    |

# [결과분석] 목적함수 값 비교 – 변경된 유입량 적용 (Case2)

## ■ Scheduling Period 별 TSD값 변화추이

- 기존 유입량 Scenario 적용 결과보다 전반적으로 오차율 감소  
⇒ [전제조건]  $Q(t) = 6, 12, 24, 18$  (for  $t = 1, 2, 3, 4$ )

### < Case 2 – TSD 결과 >

, , 2

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 800  | 629  | 549  | 569  | 689  |
| [2,] | 625  | 450  | 329  | 299  | 369  |
| [3,] | 500  | 325  | 200  | 129  | 164  |
| [4,] | 425  | 250  | 125  | 50   | 34   |
| [5,] | 400  | 225  | 100  | 25   | 0    |

, , 3

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1358 | 1005 | 825  | NaN  | NaN  |
| [2,] | 1174 | 765  | 535  | 480  | NaN  |
| [3,] | 1049 | 624  | 345  | 240  | 296  |
| [4,] | 974  | 549  | 254  | 100  | 125  |
| [5,] | 949  | 524  | 229  | 59   | 25   |

...

, , 6

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1049 | 803  | 683  | 678  | 789  |
| [2,] | 874  | 624  | 463  | 408  | 469  |
| [3,] | 749  | 499  | 334  | 238  | 264  |
| [4,] | 674  | 424  | 259  | 159  | 134  |
| [5,] | 649  | 399  | 234  | 134  | 100  |

...

, , 10

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1149 | 903  | 783  | 778  | 889  |
| [2,] | 974  | 724  | 563  | 508  | 569  |
| [3,] | 849  | 599  | 434  | 338  | 364  |
| [4,] | 774  | 524  | 359  | 259  | 234  |
| [5,] | 749  | 499  | 334  | 234  | 200  |

### < Case 1 – TSD 결과 >

, , 2

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 996  | 1025 | NaN  | NaN  | NaN  |
| [2,] | 725  | 675  | 725  | NaN  | NaN  |
| [3,] | 525  | 425  | 425  | 525  | NaN  |
| [4,] | 425  | 275  | 225  | 306  | NaN  |
| [5,] | 400  | 225  | 136  | 146  | 256  |

, , 3

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1805 | 1381 | 1121 | NaN  | NaN  |
| [2,] | 1621 | 1141 | 831  | 671  | NaN  |
| [3,] | 1496 | 1000 | 641  | 431  | 432  |
| [4,] | 1421 | 925  | 550  | 291  | 261  |
| [5,] | 1396 | 900  | 525  | 250  | 161  |

...

, , 6

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1190 | 1215 | NaN  | NaN  | NaN  |
| [2,] | 919  | 865  | 911  | NaN  | NaN  |
| [3,] | 719  | 615  | 611  | 711  | NaN  |
| [4,] | 619  | 465  | 411  | 492  | NaN  |
| [5,] | 594  | 415  | 322  | 332  | 442  |

...

, , 10

|      | [,1] | [,2] | [,3] | [,4] | [,5] |
|------|------|------|------|------|------|
| [1,] | 1376 | 1401 | NaN  | NaN  | NaN  |
| [2,] | 1105 | 1051 | 1097 | NaN  | NaN  |
| [3,] | 905  | 801  | 797  | 897  | NaN  |
| [4,] | 805  | 651  | 597  | 678  | NaN  |
| [5,] | 780  | 601  | 508  | 518  | 628  |

# 최적화 기법 별 특성 / 한계점 비교

# [기본개요] DP vs. LP vs. MILP

## ■ 통상적 Deterministic 최적화의 경우 3가지 기법이 존재

### • 분기한정 논리와 동적계획법의 경우 유사점 존재

⇒ [공통사항] Sub-problem으로 구분하여 최적해 도출

⇒ [비교사항] 전 단계 결과 반영 여부 (저장소 필요 - 동적계획법)

| 구분    | Bottom-UP (동적계획법)                                                                                                                                                                                                | Simplex (LP)                                                                                                                                               | 분기한정 (MILP)                                                                                                                                               |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 특이 사항 | <ul style="list-style-type: none"> <li>• 선형 / 비선형 제약으로부터 자유로움</li> <li>• 수리적 계획법에서 풀지 못하는 문제들을 해결 가능하나, 결과값이 최적일 아닐 수도 있음</li> <li>• 예) 높은 비선형성을 포함하나 규모가 작은 문제</li> </ul>                                       | <ul style="list-style-type: none"> <li>• 선형최적화를 풀어가는 Search 논리</li> <li>• Binary 변수를 포함하지 않은 Search 논리에서 주로 활용됨</li> <li>• 예) 전력수급 계획 및 기타 선형문제</li> </ul> | <ul style="list-style-type: none"> <li>• 혼합정수계획법을 풀어가는 Search 논리</li> <li>• Binary 변수를 포함한 Search 논리에서 주로 활용됨</li> <li>• 예) 정수장 또는 전력저장장치 운영계획</li> </ul> |
| 운영 논리 | <ul style="list-style-type: none"> <li>• Bottom-Up 방식으로 최적해를 Search</li> <li>• 작은 부분들 (Sub-Problem)의 해답을 활용하여 목적함수에 대한 해답을 찾아내는 방식</li> <li>• 선형계획법과 달리, 특정 Search 논리가 없기 때문에, 제약조건의 특성에 따라 다양한 해법 필요</li> </ul> | <ul style="list-style-type: none"> <li>• 임의적으로 최적해가 도출 가능할 것 같은 부분을 우선 탐색</li> <li>• 변수와 제약조건을 고려하여, 더 이상 탐색 가능한 해가 없을 때까지 탐색</li> </ul>                   | <ul style="list-style-type: none"> <li>• 임의적으로 최적해가 도출 가능할 것 같은 부분을 우선 탐색</li> <li>• 선형계획법으로 도출된 결과를 다시 정수값이 적용된 제약조건을 적용하여 최적해를 재탐색</li> </ul>           |

# [기본개요] 동적계획법 vs. MILP

## ■ Problem Size 변화에 따른 차이점 존재

- 문제의 크기가 커질 경우, MILP가 더 최적해 도출에 유리

⇒ [D P] 문제의 크기가 작으며 별도의 Solver 호출이 필요 없으며 제약조건의 Boundary가 Tight한 경우, 충분히 Optimal Solution이 도출 가능

- 모든 Scheduling Period 내 활용 가능한 발전설비들에 대한 정보가 우선 필요
- 제약조건 및 가용 가능 자원들이 증가하는 점을 고려할 시, 사실상 적용 불가
- 문제를 Sub-problem으로 구분해야함에, 전국적 종합적인 계통 분석이 힘들

⇒ [MILP] 문제의 크기가 크며, Solution 도출에 대한 제약조건의 Boundary가 넓은 경우, 유리

- 대표적 사례로 최적화 Interval의 대상이 8760시간에서 5분 또는 4초단위로 세분화 가능
- 비선형의 Lagrangian 완화법 보다 더 우수한 최적해 도출 가능

# [희망사항] 향후 계획

- Stochastic Scenario 만드는 Technique 학습 희망
  - 불확실성 Scenario 생성 방법
  - Scenario 소거 법

감 사 합 니 다